Lab2 实验报告

姓名：朱文凯
学号：23307110192

1. 实验目标

本次 Lab2 的目标是在已有五级流水线 CPU 的基础上，补齐内存读写相关功能，并通过 `make test-lab2` 测试。
根据实验要求，本次需要支持的指令包括：

- `ld` 、 `sd`
- `lb` 、 `lh` 、 `lw`
- `lbu` 、 `lhu` 、 `lwu`
- `sb` 、 `sh` 、 `sw`
- `lui`

与 Lab1 相比，这一阶段的重点不再只是“算术逻辑是否正确”，而是：

- Memory 阶段是否真的接入了数据总线
- 访存地址、写掩码和写数据是否对齐正确
- Load 数据能否按宽度正确截取和扩展
- 在 `ibus` 和 `dbus` 共享内存时，流水线是否能正确停顿并保持时序一致

2. 总体设计思路

这次实现基本遵循“先补通路，再补握手，最后修时序问题”的思路。

2.1 补齐译码与地址生成

第一步先在译码阶段补齐 Lab2 所需的控制语义。

主要工作包括：

- 为 `load` 指令产生 `mem_read` 、 `wb_sel` 、 `reg_write`
- 为 `store` 指令产生 `mem_write`

- 为 `lui` 补上写回路径
- 继续复用 ALU 的 `ADD` 完成访存地址生成，即 `rs1 + imm`

这样做的原因比较直接：如果译码阶段没有把 load/store/lui 真正接成控制信号，那么后面的 MEM 阶段即使写了很多逻辑，也没有办法被稳定驱动起来。

2.2 在 MEM 阶段建立真实访存通路

第二步是在 `core.sv` 里把数据总线真正接通。

这里我主要做了三件事：

- 根据 `funct3` 生成 `dreq.size`
- 根据地址低位和访存宽度生成 `dreq.strobe`
- 对 store 数据做左移对齐，对 load 数据做右移截取

这一部分最关键的点是要理解实验文档里关于 `dreq.addr` 、 `strobe` 和 `data` 的关系：
地址本身保留原始低位，而对齐工作主要由写数据移位和字节使能来完成。

2.3 加入共享总线下的停顿控制

由于 `ibus` 和 `dbus` 实际上访问同一块内存，所以一旦 MEM 阶段发起数据访存，就不能再让前端继续像没有访存一样往前冲。

这一步的核心处理是：

- 在访存占用总线期间暂停前端继续取指
- 在 `dresp.data_ok` 到来前，保持当前访存请求稳定
- 在访存结束释放总线时，避免把前端已经 hold 住的旧指令再次错误推进

这一部分和 Lab1 相比更像是在处理“流水线时序一致性”，而不是单纯加几个功能信号。

2.4 修正提交与旁路细节

真正调通以后，我发现 Lab2 最麻烦的问题不在 load/store 本身，而在它们对已有流水线时序的影响。

因此又补了两类修正：

- store 使用经过 forwarding 后的 `rs2` 数据，避免把旧值写回内存
- 提交信号改成“一条 WB 指令只提交一次”，避免 stall 期间 Difftest 重复或漏对比

这些问题如果只看波形表面，很容易误以为是 load/store 数据算错，但实际上根因往往在流水线推进和有效位控制上。

3. 主要实现内容

这次 Lab2 没有新开很多模块，主要还是在原有框架上做增量修改。

3.1 `core_decode.sv`

这一部分主要负责：

- 补齐 `load/store/lui` 的译码
- 继续生成 `rd` 、 `rs1` 、 `rs2` 、 `imm`
- 产生 `mem_read` 、 `mem_write` 、 `wb_sel` 、 `reg_write` 等控制信号

我保留了 Lab1 时“译码模块负责字段提取和组合控制，顶层负责时序推进”的思路，这样改动范围比较清晰，也不容易把控制逻辑全部堆到顶层。

3.2 `core.sv`

这次改动最多的文件还是 `core.sv` 。

主要新增或完善了：

- `funct3` 在流水线中的继续传递
- `dreq.valid` 、 `dreq.addr` 、 `dreq.size` 、 `dreq.strobe` 、 `dreq.data`
- load 数据的宽度选择与符号/零扩展
- 共享总线下的前端停顿与 MEM 等待逻辑
- store 数据使用 forwarding 后结果
- WB 提交只触发一次的控制

从结构上看，Lab2 仍然建立在 Lab1 的五级流水线骨架上，没有推翻重做；但从复杂度上看，CPU 已经从“以 ALU 为中心”转成了“以流水线一致性为中心”。

4. 关键问题与修复过程

4.1 起初的 MEM 阶段只是占位

最开始的问题其实很直接：虽然流水线里已经有 `MEM` 这个阶段，但 `dreq` 还是常量 0，数据总线根本没有真的发出访问请求。

这意味着：

- store 不会真正写内存
- load 也不会真正从内存读回数据
- `MEM` 阶段只是“名字存在”，功能上还是空的

修复方法就是把 `dreq` 真正接起来，并根据 `funct3` 和地址低位计算访存宽度、字节使能和对齐方式。

4.2 地址低位和写掩码容易想错

这次实验中最容易出错的一点，是把“地址对齐”和“写数据对齐”混在一起。

一开始我比较自然地想把地址低位清零，但后来重新对照实验文档和 `common.sv` 注释后发现：

- 总线的 `addr` 应该保留原始地址
- 真正的对齐要靠 `data` 左移和 `strobe` 掩码完成

这个细节如果搞错，现象通常不是完全跑不动，而是：

- 某些 load/store 看起来正常
- 某些 byte/halfword 场景会悄悄写错相邻字节

所以这一点必须按总线约定来做，不能凭直觉处理。

4.3 访存一加入，前端时序就会被打乱

Lab1 里只需要关注取指等待，而 Lab2 里开始有 `dbus` 访问后，问题明显复杂了很多。

因为 `ibus` 和 `dbus` 共用同一块内存，所以一旦 MEM 阶段发起数据请求，就必须考虑：

- 前端是否继续取指
- PC 是否还应该递增
- `IF_ID` 、 `ID_EX` 是否应该保持
- `EX/MEM` 在释放总线那一拍会不会重复推进旧指令

调试时我遇到过的典型现象包括：

- 指令被跳过
- 同一条指令被执行两次
- Difftest 报 PC 对不上，但 ALU 本身并没有明显错误

最后的修正思路是：

- 访存占用总线期间冻结前端
- 访存完成释放总线时对前级插入 bubble
- 保证共享总线下，前后级的状态切换是一次性的

4.4 Difftest 问题表面像“算错”，本质常常是“提交错位”

这次实验让我印象最深的一点是，Difftest 报错不一定代表数据运算逻辑错了。

我在调试中碰到过一种情况：

- 某条指令已经到达 WB
- 但因为 stall，WB 阶段被保持了多个周期
- 如果提交信号仍持续有效，就会把同一条指令重复提交给 Difftest

这样一来，后面的所有对比都会错位，看起来像寄存器值乱掉了，但根因其实是提交时序不对。

因此最后增加了“一次性提交”的控制，让一条 WB 指令只在真正提交的那个时刻对外生效。

4.5 store 也需要考虑 forwarding

一开始比较容易只关注 ALU 输入的前递，但 Lab2 里 store 写内存的数据来自 `rs2` ，如果这里仍然用寄存器堆旧值，就会出现：

- 算出来的地址对了
- 写入的数据却是旧的

所以后面我把 store 的写数据也接到了 forwarding 后的路径上。
这个修复不算很大，但如果漏掉，Lab2 会出现一些很隐蔽的错误。

5. 测试结果

最终我运行了：

```
make test-lab2
```

输出中出现了：

```
The first instruction of core 0 has commited. Difftest enabled.
Core 0: HIT GOOD TRAP at pc = 0x8001fffc
```

这说明：

- 第一条指令能够正确提交
- load/store 路径已经能和 Difftest 对齐
- 程序能够运行到正确结束位置
- 当前实现已经通过 Lab2 基本测试

从结果上看，本次 Lab2 的核心目标已经达到。

6. 反思与总结

这次实验让我最明显的感受是，随着 CPU 功能变多，真正难的部分已经不再是“把功能写出来”，而是“把功能放进流水线之后还能保持一致”。

Lab1 时，我更多在关注：

- ALU 算得对不对
- forwarding 和 stall 有没有接上

到了 Lab2，我更关注的是：

- 请求和响应之间是否保持稳定
- 不同阶段在 stall 时有没有被同时正确 hold 住
- 一条指令在流水线里是否只会被推进、提交一次

换句话说，Lab2 让我更明确地意识到：
流水线 CPU 中很多错误的根因并不在功能模块本身，而在不同阶段之间的配合方式。

这次实验也让我对后续工作有了更清楚的判断：

- 现在的数据通路已经比 Lab1 完整很多
- 但如果继续做 Lab3，重点会逐渐从 `MEM` 转向控制流
- 也就是从“会访存”进一步走向“会正确跳转、分支和冲刷”

所以我觉得 Lab2 的意义不只是把 load/store 做出来，更重要的是让我真正开始从“功能堆叠”转向“整体时序和架构一致性”的视角看流水线 CPU。

7. 大模型的使用说明

本次实验中，我使用了 Cursor / Gemini 等工具作为辅助，主要用于：

- 梳理实现阶段和调试顺序
- 对照实验要求检查实现边界
- 帮助总结关键 bug 的原因和修复思路
- 整理实验报告的结构与表达

但具体实现中最关键的部分，例如：

- `dreq` 的生成方式
- load/store 的对齐与扩展逻辑
- 共享总线下的 stall 处理
- Difftest 提交时序修复

都经过了结合代码、测试结果和实验文档的人工检查，而不是直接照搬生成内容。